

Worksheet Answers: Pretraining & Fine-tuning

CS 5090 · Software Engineering with Generative AI · answers with short explanations, for your own review. Where a question is open-ended, the answer below is one good answer, not the only one.

Class 1 · Pretraining

1.1 What is the "label" your Project 1 model learns from?

The next token itself. The corpus is its own supervision (self-supervision): for every position, the text that follows is the target. Nobody annotates anything, which is exactly why pretraining can use web-scale data.

1.2 One training example per objective from "Huskies are ready to run".

Objective	Example
Next-token (GPT)	Huskies are ready to → run (any prefix, with the following token as target)
Masked token (BERT)	Huskies are [MASK] to run → ready (any position; a mask in the middle is the better illustration, because it shows the model using context from both sides)

If you masked only the last word, you rediscovered next-token prediction: the interesting case for BERT is a mask with context on both sides.

1.3 Scaling: token budget for a 10M model, and the "more data is always better" claim.

(a) $10M \times 20 \approx 200M$ tokens.

(b) Compute-optimal scaling (Chinchilla) says parameters and tokens should grow together. 1B tokens on a 10M-parameter model is roughly $5\times$ past the optimum: the extra compute would buy more if it were spent training a larger model at about 20 tokens per parameter. Note the real-world caveat: people do deliberately overtrain small models past the compute-optimal point, because inference is cheaper forever afterwards. Compute-optimal is not the same as deployment-optimal.

1.4 Why does label-free pretraining help a task with only 5,000 labels?

Pretraining has already learned the structure of language and code from billions of free tokens, so the 5,000 labels only have to steer existing representations toward a task rather than build those representations from scratch. Starting from random weights, 5,000 examples are nowhere near enough to learn syntax, naming conventions, and API shapes as well.

Class 2 · Fine-tuning

2.1 Two ways to get a classifier from 5,000 labeled bug reports.

(i) **Fine-tune**: add a classification head (or train on task-formatted text) on the 5,000 examples. Cost: GPU training time, plus a model copy to store and version per task.

(ii) **Prompt / in-context learning**: put instructions and a few examples in the context. Cost: no training, but you pay tokens on every call and accuracy often lags a fine-tuned model on a narrow task.

(A linear probe on frozen features is a reasonable third answer. Training from scratch is the losing answer: 5,000 examples cannot rebuild what pretraining gives you for free.)

2.2 Architecture and head for four SE tasks.

Task	Architecture	New head?
Bug-report severity	encoder (decoder also works)	yes: linear classifier
Entity tagging over comments	encoder	yes: per-token classifier
Commit-message generation	encoder-decoder or decoder	no: it is a generation task
Code completion	decoder	no: already next-token

Several rows have more than one defensible answer, and the reasoning matters more than the letter. The two that are not negotiable: completion is a decoder with no new head, and token-level tagging needs a per-token head.

2.3 Full fine-tune vs LoRA on a laptop GPU; and a rising validation loss.

Plan B is the realistic one. Fully fine-tuning a 7B model needs memory for the weights *plus* gradients *plus* optimizer states,

several times the size of the weights themselves, which runs to tens of gigabytes. A laptop GPU cannot hold that. LoRA freezes the base model and trains roughly 0.1 to 1% of the parameters, which fits.

The loss curves: training loss falling while validation loss rises is overfitting. Stop early and keep the epoch-2 checkpoint; if you want to push further, reduce the learning rate or the number of epochs, get more data, or add regularization. More epochs is not automatically better.

2.4 Pretraining vs fine-tuning: data, objective, compute.

Data: web-scale unlabeled text → a small, targeted, usually labeled or task-formatted set.

Objective: self-supervised next-token prediction → a task loss, or the very same next-token loss applied to task-shaped text.

Compute: one enormous run → a comparatively tiny run, repeated once per task.

Worth remembering: in instruction tuning the loss function does not change at all. Only the shape of the data does.

Class 3 · The same ideas, in code

3.1 Predict the initial loss with a 50,257-token vocabulary.

$\approx \ln(50257) \approx 10.8$. An untrained softmax is close to uniform, so each token gets probability about $1/V$ and the cross-entropy of the correct token is $-\ln(1/V) = \ln V$.

In the notebook the printed value is a little higher, around 11. The random initialization is not perfectly uniform, so $\ln V$ behaves as a floor rather than an exact prediction. Check your own Project 1 log: with a vocabulary of 8,192 the first loss should be near $\ln(8192) \approx 9.0$.

3.2 What temperature does.

Dividing the logits by T before the softmax rescales how peaked the distribution is. $T \rightarrow 0$ sharpens it toward the single most probable token: output becomes deterministic and repetitive, effectively greedy decoding. $T = 2$ flattens it: more diverse output, and more incoherence and errors.

3.3 The "newly initialized" warning: is something broken?

Nothing is broken. The pretrained body loaded correctly; what is newly initialized is the classification head, a linear layer of shape `hidden × num_labels` that has no pretrained counterpart because the pretrained checkpoint was never trained for your five labels.

Deployed without training, the body still produces good features but the random head maps them to essentially arbitrary label scores, so you get roughly chance accuracy. This is measurable: in the LoRA notebook, accuracy on the two-class task *before* fine-tuning is about 46 to 48%, which is chance. "Pretrained" does not mean "ready for your task."

3.4 Match each code fragment to its concept.

Code fragment	Concept
<code>targets = inputs[:, 1:]</code>	(c) next-token pretraining objective
<code>x[mask_positions] = MASK_ID</code>	(d) masked-language-model objective
<code>nn.Linear(hidden, num_labels)</code>	(a) task head for fine-tuning
<code>probs = softmax(logits / T)</code>	(e) temperature decoding
<code>W_delta = B @ A, base frozen</code>	(b) LoRA

Before you leave Your own first logged loss.

It should be close to $\ln(\text{your vocabulary size})$: a vocabulary of 8,192 gives about 9.0. If your number is far off, the usual reasons are that the log starts after some training steps have already run, or that the vocabulary is not the size you assumed. Either is worth knowing before you write the model card.